# Types



## Système de types

*"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."*

Benjamin Pierce, *Types and Programming Languages*



## Système de types

*"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."*

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes



## Système de types

*"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."*

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes
- Garantit l'absence, pas la présence

```
if true then 5 else erreur
```

- Dépend du langage



## Système de types

*"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."*

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes
- Garantit l'absence, pas la présence

```
if true then 5 else erreur
```

- Dépend du langage
- Équilibre expressivité/utilisation pratique
- Souvent à la compilation (construction de l'AST)



## Utilité (en programmation)

*"Well typed programs cannot go wrong."*

Robin Milner, *A Theory of Type Polymorphism in Programming*

- Détection des erreurs
- Abstraction (modules en OCaml, ...)
- Documentation du code

## Noyau OCaml

Types

$$\tau ::= \quad \texttt{num} \qquad \text{(type des nombres)}$$
$$\texttt{str} \qquad \text{(type des chaînes)}$$

Expressions

$$e ::= \quad x \qquad \text{(variable)}$$
$$n \qquad \text{(nombre littéral)}$$
$$s \qquad \text{(chaîne littérale)}$$
$$e_1 + e_2 \qquad \text{(addition)}$$
$$e_1 \hat{} e_2 \qquad \text{(concaténation)}$$
$$\texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 \qquad \text{(définition)}$$

Terme mal formé : 10 + "hello"



## Jugements et règles de typages

- De la forme $\Gamma \vdash e : \tau$
- Environnement de typage $\Gamma, \Delta ::= x_1{:}\tau_1, \ldots, x_n{:}\tau_n$
  - sans répétition de variables : $x_1 \ldots x_n$ deux-à-deux distinctes
  - domaine $\mathrm{dom}(\Gamma) = \{x_1 \ldots x_n\}$
  - $\mathrm{fv}(e) \subseteq \mathrm{dom}(\Gamma)$



## Jugements et règles de typages

- De la forme $\Gamma \vdash e : \tau$
- Environnement de typage $\Gamma, \Delta ::= x_1{:}\tau_1, \ldots, x_n{:}\tau_n$
  - sans répétition de variables : $x_1 \ldots x_n$ deux-à-deux distinctes
  - domaine $\mathrm{dom}(\Gamma) = \{x_1 \ldots x_n\}$
  - $\mathrm{fv}(e) \subseteq \mathrm{dom}(\Gamma)$

$$\text{Num}\ \frac{}{\Gamma \vdash n : \texttt{num}} \qquad \text{Str}\ \frac{}{\Gamma \vdash s : \texttt{str}}$$

$$\text{Add}\ \frac{\Gamma \vdash e_1 : \texttt{num} \qquad \Gamma \vdash e_2 : \texttt{num}}{\Gamma \vdash e_1 + e_2 : \texttt{num}} \qquad \text{Conc}\ \frac{\Gamma \vdash e_1 : \texttt{str} \qquad \Gamma \vdash e_2 : \texttt{str}}{\Gamma \vdash e_1 \hat{} e_2 : \texttt{str}}$$



## Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)



## Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` $x$=10 `in` $x$+1.



## Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` $x$=10 `in` $x$+1.

- Est-ce que `let` $x$="*coucou*" `in` $x$ est typable ?

## Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x:\tau \vdash x:\tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1:\tau_1 \qquad \Gamma, x:\tau_1 \vdash e_2:\tau_2}{\Gamma \vdash \texttt{let}\ x{=}e_1\ \texttt{in}\ e_2:\tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` $x$=10 `in` $x$+1.

- Est-ce que `let` $x$="*coucou*" `in` $x$ est typable ? oui
- Est-ce que `let` $x$=10 `in` $x$^"*salut*" est typable ?



## Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x:\tau \vdash x:\tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1:\tau_1 \qquad \Gamma, x:\tau_1 \vdash e_2:\tau_2}{\Gamma \vdash \texttt{let}\ x{=}e_1\ \texttt{in}\ e_2:\tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` $x$=10 `in` $x$+1.

- Est-ce que `let` $x$="*coucou*" `in` $x$ est typable ? oui
- Est-ce que `let` $x$=10 `in` $x$^"*salut*" est typable ? non



## Propriétés

**Lemme (Unicité du type résultat)**
*Pour tout $e$ et $\Gamma$ tel que dom$(\Gamma) \supseteq$ fv$(e)$, il existe au plus un type $\tau$ tel que $\Gamma \vdash e:\tau$.*

**Démonstration.**
Induction sur $e$. □



## Propriétés

**Lemme (Unicité du type résultat)**
*Pour tout $e$ et $\Gamma$ tel que dom$(\Gamma) \supseteq$ fv$(e)$, il existe au plus un type $\tau$ tel que $\Gamma \vdash e:\tau$.*

**Démonstration.**
Induction sur $e$. □

**Lemme (Affaiblissement)**
*Pour tout $e$, $\Gamma$, $\tau$ tel que $\Gamma \vdash e:\tau$, pour tout $x$, $\tau'$ tel que $x \notin$ dom$(\Gamma)$, on a $\Gamma, x:\tau' \vdash e:\tau$.*

**Démonstration.**
Induction sur $\Gamma \vdash e:\tau$. □



## Propriétés

**Lemme (Substitution)**
*Pour tout $\Gamma$, $x$, $\tau_1$, $e_2$, $\tau_2$ tel que $\Gamma, x:\tau_1 \vdash e_2:\tau_2$, pour tout $e_1$ tel que $\Gamma \vdash e_1:\tau_1$, on a $\Gamma \vdash e_2\{e_1/x\}:\tau_2$.*

**Démonstration.**
Exercice □

**Lemme (Décomposition)**
*Et réciproquement*

**Démonstration.**
Exercice □



## Fonctions d'ordre supérieur

Types

| | | |
|---|---|---|
| $\tau ::=$ | `num` | (type des nombres) |
| | `func` | (type des fonctions) |

Expressions

| | | |
|---|---|---|
| $e ::=$ | $x$ | (variable) |
| | $n$ | (nombre littéral) |
| | $e_1 + e_2$ | (addition) |
| | $e_1\ e_2$ | (application) |
| | `fun` $(x{:}\tau)$ $\Rightarrow$ $e$ | (abstraction) |
| | `let` $x{=}e_1$ `in` $e_2$ | (définition) |

## Fonctions d'ordre supérieur

Types

| | | | |
|---|---|---|---|
| $\tau ::=$ | num | | (type des nombres) |
| | func | | (type des fonctions) |

Expressions

| | | | |
|---|---|---|---|
| $e ::=$ | $x$ | | (variable) |
| | $n$ | | (nombre littéral) |
| | $e_1 + e_2$ | | (addition) |
| | $e_1\ e_2$ | | (application) |
| | fun $(x{:}\tau) \Rightarrow e$ | | (abstraction) |
| | let $x{=}e_1$ in $e_2$ | | (définition) |

Distinguer
fun $(x{:}\text{num}) \Rightarrow x{+}1$ et
fun $(f{:}\text{func}) \Rightarrow$ fun $(x{:}\text{num}) \Rightarrow f\ x$



## Fonctions d'ordre supérieur

Types

| | | | |
|---|---|---|---|
| $\tau ::=$ | num | | (type des nombres) |
| | $\tau_1 \to \tau_2$ | | (type des fonctions) |

Expressions

| | | | |
|---|---|---|---|
| $e ::=$ | $x$ | | (variable) |
| | $n$ | | (nombre littéral) |
| | $e_1 + e_2$ | | (addition) |
| | $e_1\ e_2$ | | (application) |
| | fun $(x{:}\tau) \Rightarrow e$ | | (abstraction) |
| | let $x{=}e_1$ in $e_2$ | | (définition) |



## Type des fonctions

- $\tau_1 \to \tau_2$ : argument $\tau_1$, résultat $\tau_2$
- associatif à droite : $\tau_1 \to \tau_2 \to \tau_3 = \tau_1 \to (\tau_2 \to \tau_3)$

Fun
$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } (x{:}\tau_1) \Rightarrow e : \tau_1 \to \tau_2}$$

App
$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$



## Type des fonctions

- $\tau_1 \to \tau_2$ : argument $\tau_1$, résultat $\tau_2$
- associatif à droite : $\tau_1 \to \tau_2 \to \tau_3 = \tau_1 \to (\tau_2 \to \tau_3)$

Fun
$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } (x{:}\tau_1) \Rightarrow e : \tau_1 \to \tau_2}$$

App
$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

- Type de fun $(x{:}\text{num}) \Rightarrow x{+}1$ ? num → num
- Type de fun $(f{:}\text{num} \to \text{num}) \Rightarrow$ fun $(x{:}\text{num}) \Rightarrow f\ x$ ?
  (num → num) → num → num



## Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides



## Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides

Types à la Church vs à la Curry

- à la Church : $\Gamma \vdash$ fun $(x{:}\text{num}) \Rightarrow x{+}1 : \text{num} \to \text{num}$
- à la Curry : $\Gamma \vdash$ fun $x \Rightarrow x{+}1 : \text{num} \to \text{num}$

## Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides

Types à la Church vs à la Curry

- à la Church : $\Gamma \vdash \texttt{fun}\ (x{:}\mathsf{num}) \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$
- à la Curry : $\Gamma \vdash \texttt{fun}\ x \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$

Est-ce que l'unicité du type résultat est toujours valide dans le langage à la Curry ?